



LAPORAN PRAKTIKUM

Pengembangan Aplikasi Web II

Disusun oleh:

Nama :Muhammad Ilham Fajri

NIM : 2428240082

Kelas : SI5B

PROGRAM STUDI SISTEM INFORMASI
UNIVERSITAS MULTI DATA PALEMBANG
TAHUN AKADEMIK 2026/2027

1. Tujuan Praktikum

- Membangun RESTful API menggunakan Node.js dan Express.js untuk mengelola resource mahasiswa.
- Menerapkan operasi CRUD menggunakan method HTTP GET, POST, PUT, dan DELETE pada endpoint /mahasiswa.
- Menerapkan route parameter berdasarkan id dan query string untuk memfilter mahasiswa berdasarkan jurusan.
- Menerapkan validasi input dan HTTP status code sesuai hasil proses request.
- Melakukan pengujian endpoint untuk mengetahui kesesuaian request, response, dan status code.

2. Dasar Teori

2.1 REST dan Resource

REST (Representational State Transfer) merupakan pendekatan perancangan layanan web yang memanfaatkan resource dan method HTTP untuk melakukan operasi terhadap data. Pada praktikum ini resource yang dikelola adalah data mahasiswa dengan endpoint utama /mahasiswa.

2.2 Method HTTP

GET digunakan untuk mengambil data, POST untuk menambahkan data baru, PUT untuk memperbarui data berdasarkan id, dan DELETE untuk menghapus data. API pada project menyediakan operasi GET seluruh data, GET berdasarkan id, POST, PUT, DELETE, dan GET dengan query string jurusan.

2.3 HTTP Status Code

Implementasi menggunakan status 200 untuk request berhasil pada GET dan PUT, 201 untuk data baru melalui POST, 400 ketika nama atau jurusan tidak diisi, 404 ketika data berdasarkan id tidak ditemukan, dan 204 ketika data berhasil dihapus.

2.4 JSON

JSON digunakan sebagai format pertukaran data pada API. `express.json()` memungkinkan server membaca request body berformat JSON. Response GET, POST, PUT, dan error dikirim melalui response HTTP sesuai implementasi route.

2.5 Express.js

Express.js digunakan sebagai framework Node.js untuk membuat server HTTP dan routing. Project juga menggunakan middleware logger, CORS, dotenv, dan express.json().

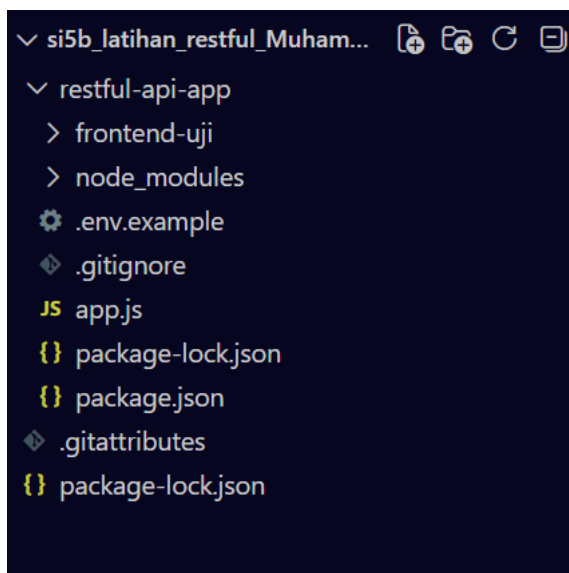
3. Alat dan Bahan

- Node.js
- Express.js
- CORS
- dotenv
- Nodemon
- Visual Studio Code atau editor kode
- Postman/Thunder Client/alat pengujian API

4. Langkah Praktikum

4.1 Struktur Project

Project pada memiliki folder utama restful-api-app yang berisi source code API, package.json, konfigurasi environment, folder frontend-uji, dan dependency Node.js.



Gambar 4.1 — Struktur project

4.2 Konfigurasi package.json

File package.json menetapkan app.js sebagai entry point serta menyediakan script start dan dev.

```
restful-api-app > {} package.json > ...
1  {
2    "name": "restful-api-app",
3    "version": "1.0.0",
4    "description": "",
5    "main": "app.js",
6    "scripts": {
7      "start": "node app.js",
8      "dev": "nodemon app.js",
9      "test": "echo \"Error: no test specified\" && exit 1"
10   },
11   "keywords": [],
12   "author": "",
13   "license": "ISC",
14   "type": "commonjs",
15   "dependencies": {
16     "cors": "^2.8.6",
17     "cors": "^2.8.6",
18     "dotenv": "^18.0.4",
19     "express": "^5.2.1",
20     "nodeman": "^1.1.2"
21   },
22   "devDependencies": {
23     "nodemon": "^3.1.14"
24   }
25 }
26
```

Gambar 4.2 — package.json

4.3 Data Awal Resource Mahasiswa

Data mahasiswa disimpan dalam array in-memory. Data awal terdiri dari dua mahasiswa dan nextId dimulai dari 3.

```
// Data sementara (disimpan di memori, hilang saat server restart)
let mahasiswa = [
  { id: 1, nama: 'Andi', jurusan: 'Sistem Informasi' },
  { id: 2, nama: 'Budi', jurusan: 'Informatika' },
];
let nextId = 3; // penghitung id untuk data baru
```

Gambar 4.3 — Data awal mahasiswa

4.4 Konfigurasi Express dan Middleware

Aplikasi menggunakan dotenv, Express, CORS, logger, dan express.json(). Logger mencatat waktu, method, dan URL setiap request.

```
restful-api-app > JS app.js > ...
1 require('dotenv').config(); // baris pertama
2 const express = require('express');
3 const cors = require('cors');
4 const app = express();
5 const PORT = process.env.PORT || 3000; // sebelumnya: const PORT = 3000;
6
7 // Middleware custom
8 // logger : mencatat setiap request yang masuk
9 function logger(req, res, next) {
10   const waktu = new Date().toISOString();
11   console.log(`[${waktu}] ${req.method} ${req.url}`);
12   next(); // wajib, agar request lanjut ke handler berikutnya
13 }
14
15 // Didaftarkan sebelum route agar mencatat seluruh request
16 app.use(logger);
17
18 app.get('/', (req, res) => {
19   res.send('Server Express.js berjalan!');
20 });
21
22 app.get('/profil', (req, res) => {
23   res.send('ini halaman profil saya');
24 });
25
26 app.get('/hubungi', (req, res) => {
27   res.send('ini halaman hubungi saya');
28 });
29 app.use(logger);
30 app.use(cors({
31   origin: process.env.CORS_ORIGIN,
32   methods: ['GET', 'POST', 'PUT', 'DELETE'],
33 }));
34 app.use(express.json());
35 // Middleware agar req.body (JSON) dapat dibaca
36 app.use(express.json());
37
```

Gambar 4.4 — Middleware aplikasi

4.5 GET Seluruh Data dan Filter

GET /mahasiswa mengembalikan seluruh data. Jika query jurusan diberikan, API memfilter data berdasarkan jurusan.

```
// GET /mahasiswa -> menampilkan seluruh data
app.get('/mahasiswa', (req, res) => {
  const {jurusan} = req.query;

  if(jurusan){
    const hasil = mahasiswa.filter((m) =>
      m.jurusan === jurusan);
    return res.json(hasil);
  }

  res.json(mahasiswa);
});
```

Gambar 4.5 — GET /mahasiswa

4.6 GET Berdasarkan ID

GET /mahasiswa/:id mengambil data berdasarkan route parameter id. Jika data tidak ditemukan, response berstatus 404.

```
// GET /mahasiswa/:id -> menampilkan satu data berdasarkan id
app.get('/mahasiswa/:id', (req, res) => {
  const id = parseInt(req.params.id);
  const data = mahasiswa.find((m) => m.id === id);

  if (!data) return res.status(404).json({ message: 'Data tidak ditemukan' });
  res.json(data);
});
```

Gambar 4.6 — GET berdasarkan ID

4.7 POST

POST /mahasiswa digunakan untuk menambahkan mahasiswa baru. Nama dan jurusan wajib diisi, sedangkan id dibuat otomatis.

```
56 // POST /mahasiswa
57 // Body: { "nama": "Citra", "jurusan": "Sistem Informasi" }
58 app.post('/mahasiswa', (req, res) => {
59   const { nama, jurusan } = req.body;
60
61   if (!nama || !jurusan) {
62     return res.status(400).json({ message: 'nama dan jurusan wajib diisi' });
63   }
64
65   const mhsbaru = { id: nextId++, nama, jurusan };
66
67   mahasiswa.push(mhsbaru);
68   res.status(201).json(mhsbaru);
69 });
```

Gambar 4.7 — POST dan validasi

4.8 PUT

PUT /mahasiswa/:id memperbarui data mahasiswa. Implementasi mempertahankan id dan menggabungkan field yang dikirim pada body dengan data lama.

```
80
81 // PUT /mahasiswa/2
82 // Body: { "nama": "Budi Santoso", "jurusan": "Informatika" }
83 app.put('/mahasiswa/:id', (req, res) => {
84   const id = parseInt(req.params.id);
85   const index = mahasiswa.findIndex((m) => m.id === id);
86
87   if (index === -1) {
88     return res.status(404).json({ message: 'Data tidak ditemukan' });
89   }
90
91   mahasiswa[index] = { ...mahasiswa[index], ...req.body, id };
92   res.json(mahasiswa[index]);
93 });
94
```

Gambar 4.8 — PUT /mahasiswa/:id

4.9 DELETE

DELETE /mahasiswa/:id menghapus data menggunakan splice. Jika berhasil, server mengembalikan status 204 No Content.

```
95 // DELETE /mahasiswa/2
96 app.delete('/mahasiswa/:id', (req, res) => {
97   const id = parseInt(req.params.id);
98   const index = mahasiswa.findIndex((m) => m.id === id);
99
100   if (index === -1) {
101     return res.status(404).json({ message: 'Data tidak ditemukan' });
102   }
103
104   mahasiswa.splice(index, 1);
105   res.status(204).send();
106 });
```

Gambar 4.9 — DELETE /mahasiswa/:id

4.10 Route Dasar dan Server

Route / menampilkan informasi bahwa server berjalan. Port menggunakan process.env.PORT atau 3000 sebagai default.

```
app.listen(PORT, () => {
  console.log(`Server berjalan di http://localhost:${PORT}`);
});
```

Gambar 4.10 — Server Express

4.11 Konfigurasi Environment

Project menyediakan .env.example yang memuat PORT, API_KEY, CORS_ORIGIN, dan DB_URI. Berdasarkan app.js, DB_URI belum digunakan dalam proses penyimpanan data.

```
restful-api-app > .env.example
1  PORT=3000
2  API_KEY=isi-dengan-kunci-anda
3  CORS_ORIGIN=http://localhost:5173
4  DB_URI=mongodb://localhost:27017/paw2
```

Gambar 4.11 — .env.example

4.12 Frontend Uji

Project menyediakan frontend-uji/index.html untuk melakukan request fetch ke endpoint mahasiswa. URL yang digunakan pada file frontend adalah localhost port 4000.

```
restful-api-app > frontend-uji > index.html > ...
1  <!DOCTYPE html>
2  <html lang="id">
3  <head>
4  |   <meta charset="UTF-8">
5  |   <title>Uji CORS</title>
6  </head>
7  <body>
8  |   <h1>Uji CORS</h1>
9  |   <button id="tombol">Panggil API</button>
10 |   <pre id="hasil"></pre>
11
12 |   <script>
13 |       document.getElementById('tombol').addEventListener('click', async () => {
14 |           const hasil = document.getElementById('hasil');
15 |           try {
16 |               const res = await fetch('http://localhost:4000/mahasiswa');
17 |               hasil.textContent = JSON.stringify(await res.json(), null, 2);
18 |           } catch (err) {
19 |               hasil.textContent = 'Gagal: ' + err.message + ' (buka Console untuk detail)';
20 |           }
21 |       });
22 |   </script>
23 </body>
24 </html>
```

Gambar 4.12 — frontend-uji/index.html

4.13 Repository dan Deployment

Setelah proses implementasi RESTful API selesai, source code project diunggah ke repository GitHub. Repository digunakan sebagai tempat penyimpanan dan version control source code. Repository dibuat dalam kondisi publik agar dapat diperiksa dan diakses sesuai kebutuhan tugas.

Repository GitHub:

[https://github.com/IlhamFajri/si5b_latihan_restful_MuhammadIlhamFajri-2428240082-.git]

Setelah source code berhasil diunggah ke GitHub, project kemudian dihubungkan dengan Vercel untuk proses deployment. Vercel digunakan untuk menjalankan RESTful API secara online sehingga endpoint dapat diakses melalui URL deployment.

URL Deployment Vercel:

[<https://si5b-latihan-restful-muhammad-ilham.vercel.app/>]

Setelah deployment berhasil, endpoint RESTful API diuji kembali menggunakan URL Vercel untuk memastikan API dapat berjalan tidak hanya pada lingkungan lokal, tetapi juga pada lingkungan deployment.

Contoh endpoint setelah deployment:

GET <https://si5b-latihan-restful-muhammad-ilham.vercel.app/mahasiswa>

GET <https://si5b-latihan-restful-muhammad-ilham.vercel.app/mahasiswa/1>

POST <https://si5b-latihan-restful-muhammad-ilham.vercel.app/mahasiswa>

PUT <https://si5b-latihan-restful-muhammad-ilham.vercel.app/mahasiswa/1>

DELETE <https://si5b-latihan-restful-muhammad-ilham.vercel.app/mahasiswa/1>

GET <https://si5b-latihan-restful-muhammad-ilham.vercel.app/mahasiswa?jurusan=Informatika>

No	Method	Endpoint	Request	Status	Hasil
1	GET	/mahasiswa	—	200	Data

					mahasiswa berhasil diambil.
2	GET	/mahasiswa/1	—	200	Data id 1 ditemukan.
3	GET	/mahasiswa/99	—	404	Data tidak ditemukan.
4	GET	/mahasiswa?jurusan=Informatika	—	200	Data difilter berdasarkan jurusan.
5	POST	/mahasiswa	{"nama":"Citra","jurusan":"Sistem Informasi"}	201	Data baru berhasil ditambahkan.
6	POST	/mahasiswa	{"nama":""}	400	Validasi nama/jurusan wajib diisi.
7	PUT	/mahasiswa/2	{"nama":"Budi Santoso","jurusan":"Informatika"}	200	Data id 2 berhasil diperbarui.
8	PUT	/mahasiswa/99	{"nama":"X","jurusan":"Y"}	404	Data tidak ditemukan.
9	DELETE	/mahasiswa/2	—	204	Data id 2 berhasil dihapus.
10	DELETE	/mahasiswa/99	—	404	Data tidak ditemukan.

6. Pembahasan

6.1 Implementasi CRUD

Implementasi telah menyediakan fungsi CRUD utama melalui method GET, POST, PUT, dan DELETE. Penggunaan route parameter id memudahkan operasi terhadap satu data, sedangkan query string jurusan digunakan untuk filtering.

6.2 Validasi

Validasi POST memastikan field nama dan jurusan tidak kosong. PUT hanya melakukan pengecekan keberadaan id dan tidak memiliki validasi field wajib seperti POST.

6.3 Status Code

Status 200 digunakan pada GET dan PUT yang berhasil, 201 pada POST berhasil, 400 pada input POST yang tidak lengkap, 404 pada id yang tidak ditemukan, dan 204 pada DELETE yang berhasil.

6.4 Keterbatasan

- Data masih in-memory dan tidak tersimpan secara permanen.
- DB_URI tersedia pada konfigurasi contoh tetapi belum digunakan oleh app.js.
- Belum ada middleware khusus untuk response JSON ketika endpoint tidak ditemukan.
- Middleware logger dan `express.json()` pada source code terdaftar lebih dari satu kali.
- Frontend uji menggunakan port 4000 sedangkan API default menggunakan port 3000.

7. Kesimpulan

Project menghasilkan RESTful API berbasis Express.js untuk resource mahasiswa dengan endpoint `/mahasiswa`. API mendukung pengambilan seluruh data, pencarian berdasarkan id, penambahan, pembaruan, penghapusan, dan filtering berdasarkan jurusan. Berdasarkan source code, operasi CRUD utama telah memiliki penanganan status HTTP 200, 201, 400, 404, dan 204.

Pengembangan berikutnya dapat dilakukan dengan menggunakan database agar data persisten, menambahkan middleware 404 JSON, merapikan middleware yang terdaftar ganda, serta menyelaraskan konfigurasi frontend dan server.

8. Lampiran

A. Repository GitHub

[https://github.com/IlhamFajri/si5b_latihan_restful_MuhammadIlhamFajri-2428240082-.git]

B. Deployment

[<https://si5b-latihan-restful-muhammad-ilham.vercel.app/>]

. Riwayat Commit

Commits on Sep 29, 2026		
push	IlhamFajri committed yesterday · 1 / 1	c4ed21b  
push	IlhamFajri committed yesterday · 1 / 1	60dfcad  
push	IlhamFajri committed yesterday · 1 / 1	2ddb09  
Initial commit	IlhamFajri committed yesterday	92c5ead  